DBMS MINI-PROJECT REPORT

CAMPUS LEDGER

Student Academic & Activity Ledger

Submitted by: Om Swati Sanjay Nimbalkar
Roll Number: 18
Department: ECE
Subject: Database Management Systems (DBMS)
Faculty Guide: Prof. Tejas Sapkal
Academic Year: 2026–2027
Submission Date: [18-09-2026]

CAMPUS LEDGER

Student Academic & Activity Ledger

DBMS Mini-Project Report

Submitted by: [Student Name(s)]

Roll Number: [Roll Number(s)]

Class: [Class / Division]

Department: [Department Name]

College: [College Name]

Academic Year: 2026–2027

---

1. INTRODUCTION

Campus Ledger is a student academic and activity management system designed to maintain student information, educational history, semester-wise results, subject marks, academic gaps, seminars, and extracurricular activities.

The system uses a relational database to organize student records and provides a graphical user interface for viewing and managing information. It also supports exporting records to Google Sheets when the integration is configured.

2. OBJECTIVES

The main objectives of Campus Ledger are:

1. To maintain student academic records in a centralized database.
2. To store previous education details such as SSC, HSC, and Diploma.
3. To manage semester-wise results and subject marks.
4. To track academic gaps and drop years.
5. To maintain seminar and activity records.
6. To search student records by name or roll number.
7. To visualize academic performance through SGPA/CGPA graphs.
8. To support Google Sheets export for reporting.

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM recommended
- Internet connection for Google Sheets integration

3.2 Software Requirements

- Operating System: Windows, Linux, or macOS
- Node.js 18 or later
- npm
- SQLite
- Web browser
- Google Sheets API credentials for spreadsheet integration

4. SYSTEM ARCHITECTURE

Campus Ledger follows a client-server architecture.

The frontend is developed using HTML, CSS, and JavaScript. The backend uses Node.js and Express to process requests and communicate with the SQLite database. Google Sheets integration allows records to be exported to a spreadsheet.

Architecture flow:

User → Web Interface → Node.js/Express Backend → SQLite Database → Google Sheets Export

Figure 4.1: System Architecture Diagram

Insert a system architecture diagram here.

5. DATABASE DESIGN

Campus Ledger uses SQLite as its relational database and primary source of truth. Related information is organized into tables to reduce duplication and maintain relationships between records.

5.1 Main Database Tables

The logical entities maintained by the system include:

1. **Student Records:** Stores student identification and profile details.
2. **Education Records:** Stores SSC, HSC, and Diploma details where applicable.
3. **Semester Results:** Stores semester-wise results and SGPA.
4. **Subject Marks:** Stores theory, internal, external, practical, and laboratory marks where applicable.
5. **Academic Gaps:** Stores available academic gap or drop-year information.
6. **Seminar Records:** Stores internal and external seminar participation.
7. **Activity Records:** Stores technical, co-curricular, and extracurricular activities.

Use the exact table names from your actual schema.sql file.

5.2 Primary Keys

Primary keys uniquely identify records in a table and prevent duplicate identifiers.

5.3 Foreign Keys and Relationships

Foreign keys connect related records. For example, semester results are linked to students, and subject marks are linked to their corresponding semester records.

Figure 5.1: Entity Relationship Diagram (ERD)

Insert the ER diagram showing tables, primary keys, foreign keys, and relationships.

5.4 Database Constraints

- **PRIMARY KEY:** Uniquely identifies records.
- **FOREIGN KEY:** Maintains relationships between tables.
- **UNIQUE:** Prevents duplicate values in configured fields.
- **CHECK:** Validates values, such as non-negative marks and SGPA between 0 and 10.
- **ON DELETE CASCADE:** Deletes dependent records when configured.
- **ON DELETE SET NULL:** Sets the related foreign key to NULL when configured and permitted.

5.5 Database Normalization

Normalization reduces data redundancy and prevents insertion, update, and deletion anomalies.

- **1NF:** Each field contains a single value.
- **2NF:** Every non-key attribute depends on the entire candidate key.
- **3NF:** Non-key attributes do not depend transitively on a candidate key.

The database separates student information from education, semester, and activity records. The actual schema should be checked to confirm that all tables satisfy 3NF.

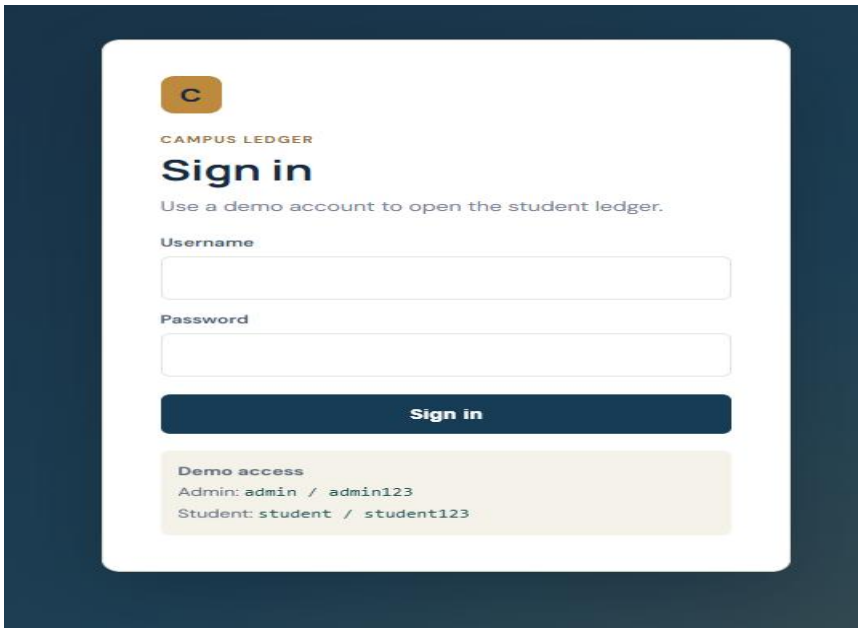
5.6 Database Storage

The application uses SQLite through the better-sqlite3 library. The schema is defined in schema.sql, and the default database file is data/ledger.db.

6. SYSTEM IMPLEMENTATION AND UI SCREENSHOTS

The system provides a graphical interface for managing and viewing student academic and activity records.

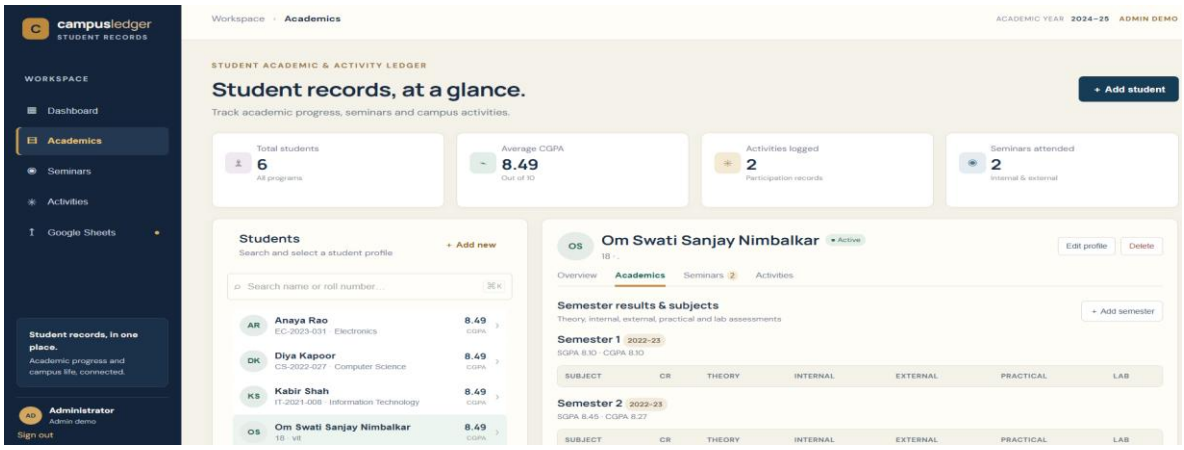
6.1 Login Page



6.1: Login Page.

Displays the login interface for accessing the system.

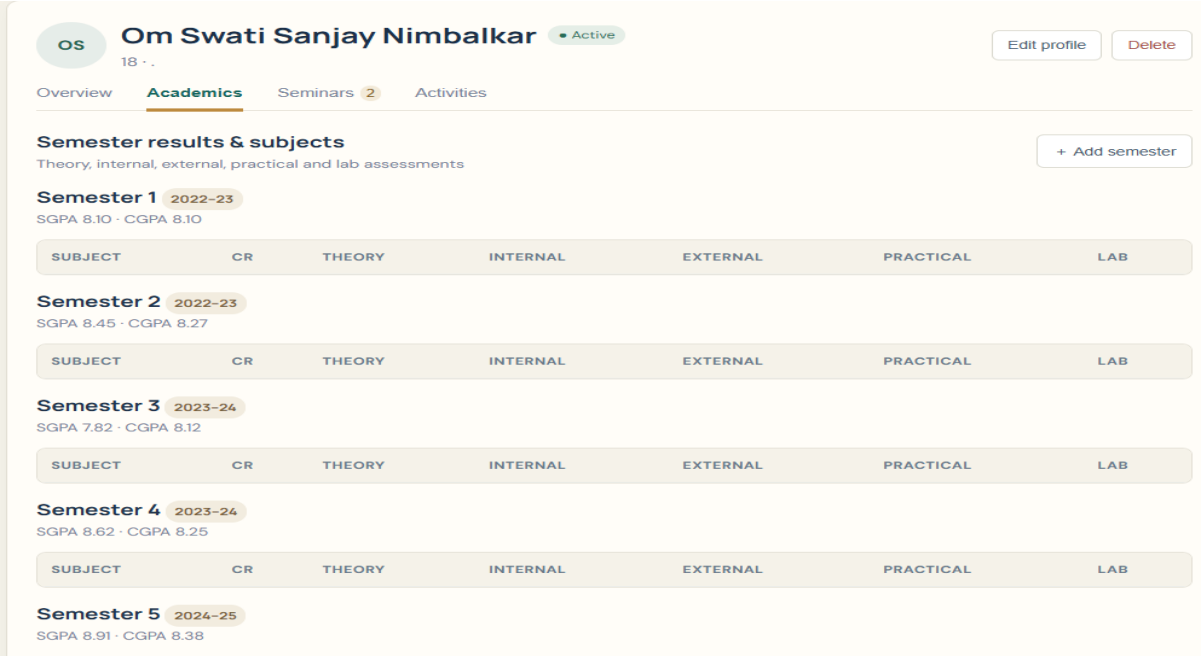
6.2 Admin Dashboard and Student Records



6.2: Admin Dashboard.

Displays student records and available management and search functions.

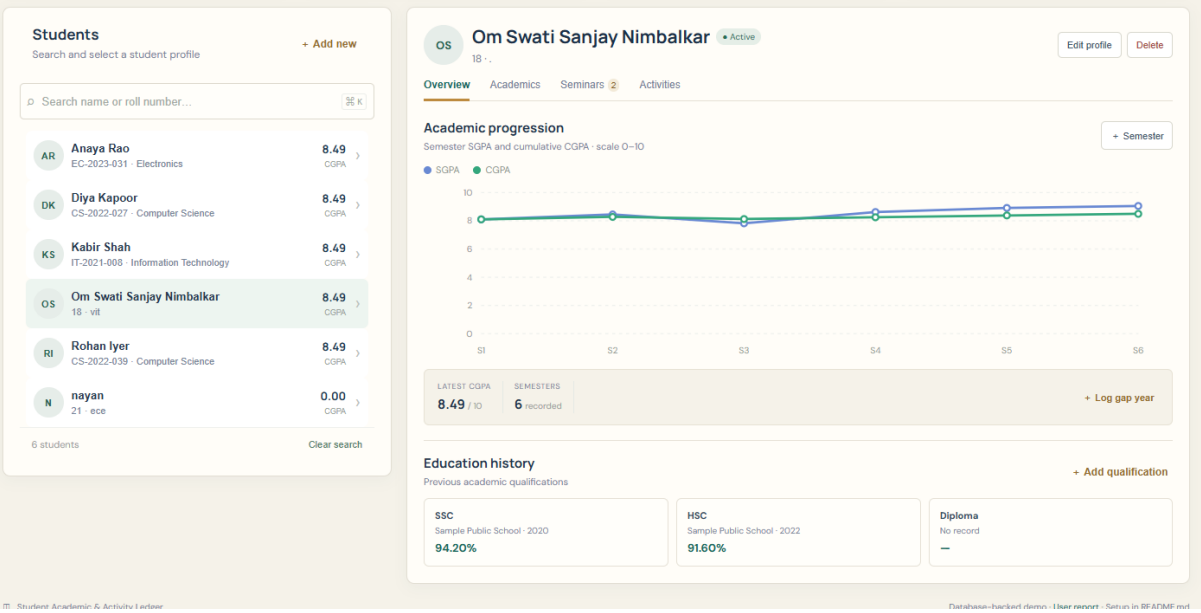
6.3 Student Profile and Education Details



6.3: Student Profile.

Displays student information and previous educational records.

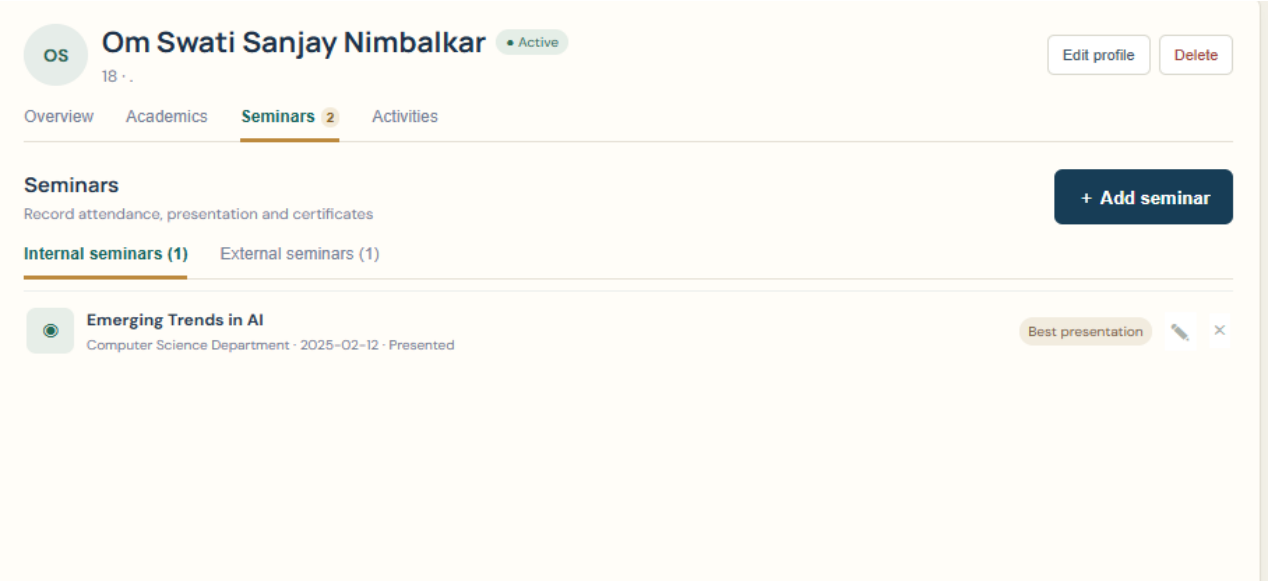
6.4 Semester Results and Performance Graph



6.4: Semester Results and SGPA/CGPA Graph.

Displays academic results and performance progression where implemented.

6.5 Seminars and Activities



6.5: Seminar and Activity Records.

Displays recorded seminar participation and student activities.

7. TESTING AND RESULTS

Testing is performed to verify that the application stores, retrieves, and manages records correctly.

Test Case	Expected Result
Student search	Matching records are displayed
Add student record	Record is saved
Update record	Changes are saved
Delete parent record	Configured foreign-key actions are applied
Invalid marks	Invalid values are rejected
Invalid SGPA	Values outside 0–10 are rejected
Google Sheets export	Records are exported when configured

Result: The application includes database integrity tests. Run npm test and record the actual results before submitting this section.

8. USER MANUAL / OPERATIONAL GUIDE

8.1 Starting the Application

1. Install Node.js 18 or later.
2. Open a terminal in the project folder.

3. Run npm install.
4. Run npm start.
5. Open http://localhost:3000 in a web browser.

8.2 Using the System

1. Log in using an available demonstration account.
2. Search for a student by name or roll number.
3. View or manage student information according to the user's role.
4. Access education, semester, seminar, and activity records.
5. Use the Google Sheets export function when configured.

Demonstration accounts: The project README lists admin / admin123 and student / student123. These credentials are for local demonstration only and must not be used in a public deployment.

9. GOOGLE SHEETS INTEGRATION

Campus Ledger supports optional one-way export of ledger records to Google Sheets through the Google Sheets API.

The integration requires Google service-account credentials and the spreadsheet ID to be configured. The spreadsheet uses a worksheet named Ledger.

SQLite remains the primary data source. Application-triggered synchronization can export records, and an administrator can retry synchronization if necessary.

Limitation: Direct database changes do not automatically trigger synchronization in the current implementation. A manual synchronization may be required. Spreadsheet edits are not imported back into SQLite.

Insert a screenshot of the actual spreadsheet and synchronization status if available.

10. LIMITATIONS AND FUTURE SCOPE

10.1 Limitations

1. Google Sheets integration requires configuration.
2. Direct SQL changes do not automatically trigger spreadsheet synchronization.
3. Spreadsheet changes are not imported into the database.
4. SGPA is entered rather than calculated from subject marks.
5. Demonstration credentials are not suitable for public deployment.

10.2 Future Scope

1. Implement automatic synchronization for direct database changes.
2. Add validated two-way synchronization if required.
3. Calculate academic performance using the applicable credit system.
4. Improve authentication and role-based access security.
5. Add advanced reports and data visualization.

11. CONCLUSION

Campus Ledger provides a centralized way to manage student academic and activity records using a relational database. It organizes information into related entities, applies database constraints, and provides a graphical interface for accessing records.

The project demonstrates the practical use of DBMS concepts, including primary keys, foreign keys, normalization, validation, and database integration.

12. REFERENCES

1. SQLite Documentation — https://www.sqlite.org/docs.html
2. Node.js Documentation — https://nodejs.org/docs/
3. Express.js Documentation — https://expressjs.com/
4. Google Sheets API Documentation — https://developers.google.com/workspace/sheets/api
5. HTML Documentation — https://developer.mozilla.org/en-US/docs/Web/HTML
6. CSS Documentation — https://developer.mozilla.org/en-US/docs/Web/CSS
7. JavaScript Documentation — https://developer.mozilla.org/en-US/docs/Web/JavaScript